CODE SPORT

Sandya Mannarswamy

This month's column features some questions on algorithms and data structures.

In the last few columns, we have discussed file system concepts. This month, we take a break from that and explore a few interview questions.

1. You are given '*N*' integers and are asked to find the second smallest element among them. How many comparisons does it take, in the worst case, to find this out?
2. A cyclic rotation of a string is obtained by chopping off a prefix of the string and attaching it to the end. For example, '*esportcod*' is a cyclic rotation of the string 'codesport'. You are given two strings P[1…n] and T[1..m]. Write an algorithm to determine whether P is a cyclic rotation of T.
3. You are given a set '*S*' of *N* integers. You are not allowed to compare two integers directly to determine which is larger. However, you are given a black box that can determine the median of a set of three integers. You are asked to write a program to determine the pair of the largest and smallest elements in *S*. What is the complexity of your program? Can you come up with an O(N) algorithm to solve this problem? Now, if you are asked to sort the set *S*, what would be the complexity of your program?
4. You are given a set of '*k*' nuts and k bolts. For each bolt present in the collection, there is exactly one nut that matches the bolt. You are not allowed to compare one bolt with another bolt or one nut with another. However, you can compare a nut and a bolt to see whether they match or not. You are asked to find the largest and smallest bolts. What is the maximum number of comparisons you need to make, in the worst case?
5. You are given two sorted arrays '*A*' and '*B*', each containing *N* elements. You are also given an integer '*k*' such that k < 2N. You are asked to find the '*k*'th smallest element in the union of the two arrays *A* and *B*. What is the complexity of your program? Assume that the arrays '*A*' and '*B*' do not contain any duplicates.
6. All of us are familiar with the utility 'make', which typically is used to compile source files into an executable. The 'make' utility reads a file called 'Makefile', which specifies the rules and dependencies. The Makefile specifies the list of source files that need to be compiled and the dependencies. The basic Makefile typically looks like what follows:

```
target: dependencies
[tab] system command
```

For instance, if there are two files, say, *hello.c* and *factorial.c*, and you want to build an executable 'prog1', here is how your Makefile would look:

```
all: prog1

hello: factorial.o hello.o
      g++ factorial.o hello.o -o prog1

hello.o: hello.c
      gcc -c hello.c

factorial.o: factorial.c
      gcc -c factorial.c
clean:
      rm -rf *.o  prog1
```

So if the programmer changes *hello.c*, then when *'make'* is invoked, it understands from the Makefile that the target executable *'prog1'* is